```
================================================================================
📦 Estrutura de diretórios do DrTilapia
├── backend
│   ├── app
│   │   ├── auth
│   │   │   ├── __init__.py
│   │   │   └── auth_service.py
│   │   ├── services
│   │   │   ├── __init__.py
│   │   │   └── rag_service.py
│   │   ├── __init__.py
│   │   └── database.py
│   ├── docs
│   │   ├── BIA_RAG.docx
│   │   └── BIA_RAG.pdf
│   ├── vectorstore_faiss
│   │   ├── index.faiss
│   │   └── index.pkl
│   ├── create_standard_user.py
│   ├── criar_admin.py
│   ├── criar_usuario_admin.py
│   ├── requirements.txt
│   ├── server.py
│   ├── setup_test_user.py
│   ├── test_auth.py
│   ├── test_auth_detailed.py
│   ├── test_authenticate_user.py
│   ├── test_connection.py
│   ├── test_integration.py
│   ├── test_login_request.py
│   └── test_rag.py
├── frontend
│   ├── assets
│   │   ├── bg_hero.png
│   │   ├── bg_hero_old.png
│   │   ├── favicon.ico
│   │   ├── Hub.jpeg
│   │   ├── hub01.jpeg
│   │   └── ImagemPrincipal.png
│   ├── ui
│   │   ├── pages
│   │   │   ├── admin_rag.py
│   │   │   ├── consultoria.py
│   │   │   ├── dashboard.py
│   │   │   ├── hub.py
│   │   │   ├── index.py
│   │   │   └── login.py
│   │   ├── __init__.py
│   │   ├── state.py
│   │   ├── styles.py
│   │   └── ui.py
│   ├── uploaded_files
│   ├── .gitignore
│   ├── requirements.txt
```

```
│   └── rxconfig.py
├── .gitignore
├── AnaliseClaude260331.pdf
├── Dockerfile
├── DrTilapIA_relatorio_completo.txt
├── Estrutura Sisitema 260402.docx
├── Estrutura Sisitema 260402.pdf
├── Estrutura_dir.py
├── GuiaIMplementação260402.pdf
├── Retorna_conteudo.py
└── Supabase
```

Arquivos que compõe o sistema DrTilapia com o nome do arquivo logo após a palavra "Arquivo:" e seu conteúdo após a palavra "Conteúdo:"

Diretório: C:\Users\usuario\Python\DrTilapIA

Arquivo: Estrutura_dir.py
Conteúdo:

```
from pathlib import Path

# Diretórios considerados "lixo" em projetos reais
IGNORAR_DIRS = {
    ".git",
    ".idea",
    ".vscode",
    "__pycache__",
    "node_modules",
    ".pytest_cache",
    "dist",
    "build",
    "venv",
    "env",
}

# Arquivos inúteis também podem ser ignorados aqui
IGNORAR_ARQUIVOS = {
    ".DS_Store",
    "Thumbs.db",
}


def gerar_arvore(diretorio, prefixo=""):
    caminho = Path(diretorio)

    # Lista somente itens relevantes
    itens = []
    for item in caminho.iterdir():

        # Ignorar ocultos (.qualquercoisa)
        if item.name.startswith(".") and item.name != ".gitignore":
            continue
```

```
        # Ignorar diretórios lixo
        if item.is_dir() and item.name in IGNORAR_DIRS:
            continue

        # Ignorar arquivos lixo
        if item.is_file() and item.name in IGNORAR_ARQUIVOS:
            continue

        itens.append(item)

    # Ordenação: pastas primeiro, depois arquivos
    itens.sort(key=lambda x: (not x.is_dir(), x.name.lower()))

    total = len(itens)
    for index, item in enumerate(itens):
        ultimo = (index == total - 1)
        conector = "└── " if ultimo else "├── "

        print(prefixo + conector + item.name)

        if item.is_dir():
            novo_prefixo = prefixo + ("    " if ultimo else "│   ")
            gerar_arvore(item, novo_prefixo)


# --- Execução ---
print("📦 Estrutura do Projeto (clean):")
gerar_arvore(".")
==================================================

Arquivo: Retorna_conteudo.py
Conteúdo:
import os


def gerar_relatorio(diretorio_raiz, arquivo_saida, extensoes=None):
    """
    extensoes: tupla/lista de extensões, ex: ('.py', '.js', '.ts')
    se extensoes for None, pega todos os arquivos
    """
    if extensoes is not None:
        extensoes = tuple(extensoes)

    with open(arquivo_saida, 'w', encoding='utf-8') as f:
        f.write(f"Conteúdo completo do diretório:
{os.path.basename(diretorio_raiz)}\n")
        f.write("=" * 80 + "\n\n")

        for root, dirs, files in os.walk(diretorio_raiz):
            # Ignora diretórios que começam com "."
            dirs[:] = [d for d in dirs if not d.startswith('.')]

            rel_path = os.path.relpath(root, diretorio_raiz)
```

```
            f.write(f"Diretório: {rel_path if rel_path != '.' else
diretorio_raiz}\n")
            f.write("-" * 50 + "\n")

            for file in sorted(files):
                if extensoes is not None and not file.endswith(extensoes):
                    continue

                caminho_completo = os.path.join(root, file)
                caminho_rel = os.path.relpath(caminho_completo, diretorio_raiz)
                f.write(f"\nArquivo: {caminho_rel}\n")
                f.write("Conteúdo:\n")
                try:
                    with open(caminho_completo, 'r', encoding='utf-8') as
arquivo:
                        conteudo = arquivo.read()
                        f.write(conteudo)
                except UnicodeDecodeError:
                    f.write("[Erro: arquivo binário ou encoding inválido -
conteúdo omitido]\n")
                except Exception as e:
                    f.write(f"[Erro ao ler: {str(e)}]\n")
                f.write("\n" + "=" * 50 + "\n")


# Uso
diretorio = r'C:\Users\usuario\Python\DrTilapIA'
extensoes_desejadas = ('.py',)
nome_saida = f"{os.path.basename(diretorio)}_relatorio_completo.txt"

gerar_relatorio(diretorio, nome_saida, extensoes=extensoes_desejadas)
print(f"Relatório gerado: {nome_saida}")
==================================================
Diretório: backend
--------------------------------------------------

Arquivo: backend\create_standard_user.py
Conteúdo:
import bcrypt
from app.database import supabase


def create_user(email, password, name, role):
    # Gerando Hash seguro com Bcrypt
    salt = bcrypt.gensalt()
    hashed = bcrypt.hashpw(password.encode('utf-8'), salt).decode('utf-8')

    data = {
        "email": email,
        "hashed_password": hashed,
        "full_name": name,
        "role": role  # 'admin' ou 'user'
    }
```

```
    try:
        # Insere na tabela public.users
        res = supabase.table("users").insert(data).execute()
        print(f"☑ Usuário {email} criado com sucesso como '{role}'!")
    except Exception as e:
        print(f"❌ Erro ao criar usuário: {e}")


if __name__ == "__main__":
    # Criando o usuário padrão solicitado
    create_user("usuario@gmail.com", "123456", "Usuário Comum", "user")
==================================================

Arquivo: backend\criar_admin.py
Conteúdo:
import os
import sys

# 1. Localiza a pasta 'backend' de forma absoluta
# Isso evita problemas com o OneDrive e caminhos relativos no Windows
BASE_DIR = os.path.dirname(os.path.abspath(__file__))

# 2. Injeta o caminho no sys.path para o Python "enxergar" a pasta app
if BASE_DIR not in sys.path:
    sys.path.insert(0, BASE_DIR)

try:
    # Importação direta (agora o Python sabe que 'app' está na raiz do sys.path)
    from app.database import supabase
    from app.auth.auth_service import get_password_hash


    def main():
        print("\n" + "=" * 50)
        print("   DR. TILÁPIA - CONFIGURAÇÃO DE ACESSO ADMIN")
        print("=" * 50)

        email = input("📧 E-mail do Administrador: ").strip()
        senha = input("🔑 Senha do Administrador: ").strip()

        if not email or not senha:
            print("❌ Erro: Campos obrigatórios vazios.")
            return

        print("\n⏳ Gerando Hash e conectando ao Supabase...")

        try:
            # Gerando o hash usando sua função do auth_service
            senha_hash = get_password_hash(senha)

            # Inserindo na tabela 'users'
            # IMPORTANTE: Verifique se as colunas no Supabase são exatamente
estas
            dados = {
```

```
                "email": email,
                "hashed_password": senha_hash,
                "role": "admin"
            }

            response = supabase.table("users").insert(dados).execute()

            if response.data:
                print(f"\n✅ SUCESSO! Usuário '{email}' criado.")
                print("🚀 Agora você pode logar no Frontend (Porta 3000).")
            else:
                print("\n❌ Erro: O banco não retornou confirmação.")

        except Exception as e:
            print(f"\n❌ Erro na operação: {e}")


    if __name__ == "__main__":
        main()

except ImportError as e:
    print(f"\n❌ ERRO DE IMPORTAÇÃO: {e}")
    print(f"DEBUG: Verifique se existe o arquivo: {os.path.join(BASE_DIR, 'app',
'database.py')}")
    print(f"DEBUG: Verifique se existe o arquivo: {os.path.join(BASE_DIR, 'app',
'__init__.py')}")
==================================================

Arquivo: backend\criar_usuario_admin.py
Conteúdo:
import os
import logging
import re
from supabase import create_client, Client
from app.services.user_service import UserService
from dotenv import load_dotenv
from typing import List, Dict, Any, Optional

# Carrega variáveis de ambiente do arquivo .env
load_dotenv()

# Configuração de logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s -
%(levelname)s - %(message)s')
logger = logging.getLogger(__name__)


class AdminCLI:
    """
    Interface de linha de comando para gerenciar usuários do Supabase Auth.
    Utiliza o UserService para operações administrativas.
    """

    def __init__(self):
```

```
        try:
            self.user_service = UserService()
            logger.info("AdminCLI inicializado com UserService.")
        except ValueError as e:
            logger.critical(f"Erro de inicialização do UserService: {e}")
            print(f"\n❌ ERRO CRÍTICO: {e}")
            print(
                "   Certifique-se de que SUPABASE_URL e
SUPABASE_SERVICE_ROLE_KEY estão definidos no seu arquivo .env.")
            exit(1)

    def _validate_email(self, email: str) -> bool:
        """Valida o formato do email."""
        if not re.fullmatch(r"[^@]+@[^@]+\.[^@]+", email):
            print("❌ Formato de email inválido.")
            return False
        return True

    def _validate_password_strength(self, password: str) -> bool:
        """Valida a força da senha (mínimo 8 caracteres, maiúscula, minúscula,
número, caractere especial)."""
        if len(password) < 8:
            print("❌ A senha deve ter no mínimo 8 caracteres.")
            return False
        if not re.search(r"[A-Z]", password):
            print("❌ A senha deve conter pelo menos uma letra maiúscula.")
            return False
        if not re.search(r"[a-z]", password):
            print("❌ A senha deve conter pelo menos uma letra minúscula.")
            return False
        if not re.search(r"\d", password):
            print("❌ A senha deve conter pelo menos um número.")
            return False
        if not re.search(r"[!@#$%^&*(),.?\"{}\|<>]", password):
            print("❌ A senha deve conter pelo menos um caractere especial.")
            return False
        return True

    def _get_user_input(self, prompt: str, sensitive: bool = False) -> str:
        """Obtém entrada do usuário, com opção para entrada sensível (senha)."""
        if sensitive:
            return input(prompt)
        return input(prompt)

    def display_menu(self):
        """Exibe o menu de opções para o usuário."""
        print("\n--- Gerenciamento de Usuários Supabase ---")
        print("1. Criar Novo Usuário")
        print("2. Listar Usuários Existentes")
        print("3. Deletar Usuário")
        print("4. Sair")
        print("")

    def handle_create_user(self):
```

```
        """Processa a criação de um novo usuário."""
        print("\n--- Criar Novo Usuário ---")
        while True:
            email = self._get_user_input("Email do novo usuário: ").strip()
            if self._validate_email(email):
                break

        existing_user = self.user_service.get_user_by_email(email)
        if existing_user:
            print(f"⚠ Usuário com email '{email}' já existe. ID:
{existing_user.get('id')}")
            return

        while True:
            password = self._get_user_input("Senha do novo usuário: ",
sensitive=True).strip()
            if self._validate_password_strength(password):
                break

        name = self._get_user_input("Nome completo do usuário: ").strip()

        roles = ["admin", "user", "guest"]
        while True:
            role = self._get_user_input(f"Role do usuário ({', '.join(roles)}):
").strip().lower()
            if role in roles:
                break
            print(f"❌ Role inválida. Escolha entre: {', '.join(roles)}")

        try:
            new_user = self.user_service.create_user(email, password, name,
role=role)
            print(f"\n🎉 Usuário '{email}' criado com sucesso!")
            print(f"   ID: {new_user.get('id')}")
            print(f"   Nome: {new_user.get('user_metadata', {}).get('name')}")
            print(f"   Email: {new_user.get('email')}")
            print(f"   Role: {new_user.get('user_metadata', {}).get('role')}")
        except Exception as e:
            logger.error(f"Erro ao criar usuário '{email}': {e}", exc_info=True)
            print(f"\n❌ ERRO ao criar usuário: {e}")

    def handle_list_users(self):
        """Processa a listagem de usuários existentes."""
        print("\n--- Listar Usuários ---")
        try:
            users = self.user_service.list_users()
            if not users:
                print("Nenhum usuário encontrado.")
                return

            print(f"Total de usuários: {len(users)}")
            for user in users:
                user_id = user.get('id', 'N/A')
                email = user.get('email', 'N/A')
```

```
                name = user.get('user_metadata', {}).get('name', 'N/A')
                role = user.get('user_metadata', {}).get('role', 'N/A')
                created_at = user.get('created_at', 'N/A')
                print(f"")
                print(f"ID: {user_id}")
                print(f"Email: {email}")
                print(f"Nome: {name}")
                print(f"Role: {role}")
                print(f"Criado em: {created_at}")
            print(f"")
        except Exception as e:
            logger.error(f"Erro ao listar usuários: {e}", exc_info=True)
            print(f"\n❌ ERRO ao listar usuários: {e}")

    def handle_delete_user(self):
        """Processa a exclusão de um usuário."""
        print("\n--- Deletar Usuário ---")
        user_id = self._get_user_input("ID do usuário a ser deletado: ").strip()
        if not user_id:
            print("❌ ID do usuário não pode ser vazio.")
            return

        try:
            user_to_delete = self.user_service.get_user_by_id(user_id)
            if not user_to_delete:
                print(f"⚠️ Usuário com ID '{user_id}' não encontrado.")
                return

            confirm = self._get_user_input(
                f"Tem certeza que deseja deletar o usuário
'{user_to_delete.get('email')}' (s/N)? ").strip().lower()
            if confirm == 's':
                self.user_service.delete_user(user_id)
                print(f"\n✅ Usuário '{user_id}' deletado com sucesso.")
            else:
                print("Operação de exclusão cancelada.")
        except Exception as e:
            logger.error(f"Erro ao deletar usuário '{user_id}': {e}",
exc_info=True)
            print(f"\n❌ ERRO ao deletar usuário: {e}")

    def run(self):
        """Executa o loop principal da interface de linha de comando."""
        while True:
            self.display_menu()
            choice = self._get_user_input("Escolha uma opção: ").strip()

            if choice == '1':
                self.handle_create_user()
            elif choice == '2':
                self.handle_list_users()
            elif choice == '3':
                self.handle_delete_user()
            elif choice == '4':
```

```
                print("Saindo do gerenciador de usuários. Até mais!")
                break
            else:
                print("Opção inválida. Por favor, tente novamente.")


if __name__ == "__main__":
    cli = AdminCLI()
    cli.run()
```

==========

Arquivo: backend\server.py
Conteúdo:

```
import logging
import os
import shutil
from pathlib import Path
from typing import List
import jwt
from dotenv import load_dotenv
from fastapi import Depends, FastAPI, HTTPException, Request, status,
UploadFile, File
from fastapi.middleware.cors import CORSMiddleware
from fastapi.security import HTTPBearer
from pydantic import BaseModel
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.errors import RateLimitExceeded
from slowapi.util import get_remote_address

# --- CONFIGURAÇÃO ---
basedir = Path(__file__).resolve().parent
load_dotenv(dotenv_path=basedir / '.env')

from app.auth.auth_service import ALGORITHM, SECRET_KEY, create_access_token,
verify_password
from app.database import supabase
from app.services.rag_service import rag_service

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -
%(message)s')
logger = logging.getLogger("DrTilapia_API")

app = FastAPI(title="Dr. Tilápia 2.0 API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
```

```
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)


class LoginRequest(BaseModel):
    email: str
    password: str


class ChatRequest(BaseModel):
    message: str
    history: List[List[str]] = []


security = HTTPBearer()


def verify_token(credentials=Depends(security)):
    token = credentials.credentials
    try:
        return jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    except:
        raise HTTPException(status_code=401, detail="Token inválido")


# --- ENDPOINTS ---

@app.post("/auth/login")
async def login(credentials: LoginRequest):
    res = supabase.table("users").select("email, hashed_password, full_name,
role").eq("email",

      credentials.email).execute()
    if not res.data or not verify_password(credentials.password,
res.data[0].get("hashed_password", "")):
        raise HTTPException(status_code=401, detail="Credenciais inválidas")

    user = res.data[0]
    token = create_access_token(data={"sub": user["email"], "role":
user.get("role", "user")})
    return {
        "access_token": token,
        "user_name": user.get("full_name"),
        "user_role": user.get("role", "user")
    }


@app.post("/consultoria/chat")
async def chat(req: ChatRequest, token: dict = Depends(verify_token)):
    return await rag_service.get_response(req.message, [tuple(h) for h in
req.history])


@app.post("/admin/upload")
async def upload(file: UploadFile = File(...), token: dict =
```

```
Depends(verify_token)):
    if token.get("role") != "admin":
        raise HTTPException(status_code=403, detail="Acesso negado")
    docs_dir = Path("docs")
    docs_dir.mkdir(exist_ok=True)
    file_path = docs_dir / file.filename.replace(" ", "_")
    with file_path.open("wb") as buffer:
        shutil.copyfileobj(file.file, buffer)
    return {"message": "Upload realizado"}


@app.post("/admin/reindex")
async def reindex(token: dict = Depends(verify_token)):
    if token.get("role") != "admin":
        raise HTTPException(status_code=403, detail="Acesso negado")
    rag_service._vectorstore = None
    rag_service._load_vectorstore()
    return {"message": "Reindexado"}


if __name__ == "__main__":
    import uvicorn

    uvicorn.run(app, host="0.0.0.0", port=8080)
==================================================

Arquivo: backend\setup_test_user.py
Conteúdo:
import os
import logging
from pathlib import Path
from dotenv import load_dotenv
from supabase import create_client, Client
from passlib.context import CryptContext

# --- CONFIGURAÇÃO DE AMBIENTE ---
basedir = Path(__file__).resolve().parent
env_path = basedir / '.env'
load_dotenv(dotenv_path=env_path)

# Configuração do Bcrypt
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

# Configuração Supabase
url = os.environ.get("SUPABASE_URL")
key = os.environ.get("SUPABASE_KEY")
supabase: Client = create_client(url, key)

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("SetupUser")


def sync_test_user():
    email = "ffasti01@gmail.com"
```

```
    password = "123456"  # ⚠️ Use a mesma senha do seu test_integration.py
    hashed_password = pwd_context.hash(password)

    print(f"\n🚀 Sincronizando usuário no Supabase (Tabela Public): {email}")

    try:
        # Verifica se o usuário existe
        res = supabase.table("users").select("id").eq("email", email).execute()

        # Mapeamento EXATO conforme o seu JSON de public.users
        user_data = {
            "email": email,
            "hashed_password": hashed_password,
            "full_name": "Fabio Ffasti"
        }

        if res.data:
            print("🔄 Usuário encontrado. Atualizando 'hashed_password'...")
            supabase.table("users").update(user_data).eq("email",
email).execute()
            print("✅ Sucesso: Hash de senha e nome atualizados.")
        else:
            print("🆕 Usuário não encontrado. Criando novo registro...")
            supabase.table("users").insert(user_data).execute()
            print("✅ Sucesso: Usuário criado com sucesso.")

        print(f"\n👉 Senha para o teste: '{password}'")

    except Exception as e:
        print(f"❌ Erro ao acessar Supabase: {e}")


if __name__ == "__main__":
    sync_test_user()
==================================================

Arquivo: backend\test_auth.py
Conteúdo:
import os
import sys
import json
from dotenv import load_dotenv
from supabase import create_client, Client
import logging

# Configuração de Logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -
%(message)s')
logger = logging.getLogger(__name__)


def run_auth_test():
    """
    Executa uma série de testes para diagnosticar problemas de autenticação com
```

```
o Supabase.
    Verifica variáveis de ambiente, conexão e tenta um login.
    """
    logger.info("Iniciando teste de autenticação do Supabase...")

    # 1. Carregar variáveis de ambiente
    load_dotenv()

    supabase_url: str = os.environ.get("SUPABASE_URL")
    supabase_key: str = os.environ.get("SUPABASE_KEY")

    # 2. Exibir as credenciais carregadas (mascaradas por segurança)
    logger.info(f"SUPABASE_URL carregada: {supabase_url is not None}")
    logger.info(f"SUPABASE_KEY carregada: {supabase_key is not None}")

    if not supabase_url or not supabase_key:
        logger.error("Erro: SUPABASE_URL ou SUPABASE_KEY não encontradas no
arquivo .env.")
        logger.error("Certifique-se de que o arquivo .env está na raiz do
diretório 'backend' e contém as chaves.")
        sys.exit(1)

    # 3. Testar a conexão com Supabase
    try:
        supabase: Client = create_client(supabase_url, supabase_key)
        logger.info("Cliente Supabase criado com sucesso.")
    except Exception as e:
        logger.error(f"Erro ao criar cliente Supabase: {e}")
        sys.exit(1)

    # Teste de conexão básica
    try:
        response = supabase.from_('users').select('id').limit(1).execute()
        logger.info(f"Conexão básica com Supabase bem-sucedida.")
    except Exception as e:
        logger.error(f"Erro ao realizar consulta de teste no Supabase: {e}")
        logger.error("Verifique se a SUPABASE_URL e SUPABASE_KEY estão
corretas.")
        sys.exit(1)

    logger.info("Conexão com Supabase verificada com sucesso.")

    # 4. Tentar fazer login com credenciais de teste
    print("\n--- Teste de Login ---")
    test_email = input("Por favor, digite o email do usuário para teste de
login: ")
    test_password = input("Por favor, digite a senha do usuário para teste de
login: ")

    if not test_email or not test_password:
        logger.warning("Email ou senha não fornecidos. Pulando teste de login.")
        return

    try:
```

```
        logger.info(f"Tentando login para o email: {test_email}")
        auth_response = supabase.auth.sign_in_with_password({
            "email": test_email,
            "password": test_password
        })

        # 5. Exibir o resultado completo do login
        if auth_response.user:
            logger.info("☑ Login bem-sucedido!")
            logger.info(f"ID do Usuário: {auth_response.user.id}")
            logger.info(f"Email do Usuário: {auth_response.user.email}")
            logger.info(
                f"Nome Completo (metadata):
{auth_response.user.user_metadata.get('full_name', 'N/A') if
auth_response.user.user_metadata else 'N/A'}")
            logger.info(f"Access Token (primeiros 20 caracteres):
{auth_response.session.access_token[:20]}...")
            logger.info("Login funcionando corretamente!")
        else:
            logger.error("✕ Login falhou. Nenhuma informação de usuário na
resposta.")

    except Exception as e:
        logger.error(f"✕ Erro durante a tentativa de login: {e}")
        logger.error(
            "Verifique se o email e a senha estão corretos e se o usuário existe
e está confirmado no Supabase.")
        logger.error(f"Tipo de erro: {type(e).__name__}")

    logger.info("Teste de autenticação concluído.")


if __name__ == "__main__":
    current_dir = os.path.dirname(os.path.abspath(__file__))
    project_root = os.path.abspath(os.path.join(current_dir, os.pardir))

    if project_root not in sys.path:
        sys.path.insert(0, project_root)

    run_auth_test()
==================================================

Arquivo: backend\test_auth_detailed.py
Conteúdo:
import os
import sys
import logging
import traceback
from dotenv import load_dotenv

# Configuração de Logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -
%(message)s')
logger = logging.getLogger(__name__)
```

```
def run_detailed_auth_test():
    """
    Executa um teste detalhado da função authenticate_user() para diagnosticar
erros.
    Captura e exibe o tipo exato da exceção, mensagem e traceback completo.
    """
    logger.info("\n--- Iniciando teste DETALHADO da função authenticate_user()
---")

    # 1. Carregar variáveis de ambiente
    logger.info("Passo 1: Carregando variáveis de ambiente do .env...")
    load_dotenv()
    logger.info("Variáveis de ambiente carregadas.")

    # Adiciona o diretório raiz do projeto ao sys.path
    current_dir = os.path.dirname(os.path.abspath(__file__))
    project_root = os.path.abspath(os.path.join(current_dir, os.pardir))
    if project_root not in sys.path:
        sys.path.insert(0, project_root)
    logger.info(f"sys.path ajustado para incluir: {project_root}")

    # 2. Importar a função authenticate_user
    logger.info("Passo 2: Tentando importar 'authenticate_user' de
'app.auth.auth_service'...")
    try:
        from app.auth.auth_service import authenticate_user
        logger.info("Função 'authenticate_user' importada com sucesso.")
    except ImportError as e:
        logger.error(f"Erro de importação: {e}")
        logger.error("Verifique se o arquivo 'app/auth/auth_service.py' existe e
está no caminho correto.")
        sys.exit(1)
    except Exception as e:
        logger.error(f"Erro inesperado ao importar 'authenticate_user': {e}",
exc_info=True)
        sys.exit(1)

    # 3. Solicitar credenciais ao usuário
    print("\n--- Credenciais para Teste ---")
    test_email = input("Por favor, digite o email do usuário para teste: ")
    test_password = input("Por favor, digite a senha do usuário para teste: ")

    if not test_email or not test_password:
        logger.warning("Email ou senha não fornecidos. Encerrando teste.")
        return

    # 4. Executar o teste de autenticação e capturar qualquer exceção
    logger.info(f"\nPasso 3: Chamando authenticate_user() para o email:
{test_email}")
    try:
        user_data = authenticate_user(test_email, test_password)

        # 5. Exibir os dados do usuário se for um sucesso
```

```
        logger.info("☑ authenticate_user() executada com sucesso!")
        logger.info("Dados do usuário autenticado:")
        for key, value in user_data.items():
            logger.info(f"  {key}: {value}")

    except Exception as e:
        # 4. Capturar QUALQUER exceção e exibir detalhes
        logger.error(f"❌ Ocorreu uma exceção durante a autenticação!")
        logger.error(f"Tipo da exceção: {type(e).__name__}")
        logger.error(f"Mensagem da exceção: {e}")
        logger.error("Traceback completo:")
        traceback.print_exc(file=sys.stderr)
        logger.error("Isso indica um problema na função authenticate_user() ou
na comunicação com o Supabase.")

    logger.info("\n--- Teste DETALHADO da função authenticate_user() concluído
---")

if __name__ == "__main__":
    run_detailed_auth_test()
==================================================

Arquivo: backend\test_authenticate_user.py
Conteúdo:
import os
import sys
import logging
from dotenv import load_dotenv

# Configuração de Logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -
%(message)s')
logger = logging.getLogger(__name__)

# --- PRIMEIRA COISA: CARREGAR VARIÁVEIS DE AMBIENTE ---
load_dotenv()
logger.info("Variáveis de ambiente carregadas.")

# Adiciona o diretório ao sys.path
current_dir = os.path.dirname(os.path.abspath(__file__))
project_root = os.path.abspath(os.path.join(current_dir, os.pardir))
if project_root not in sys.path:
    sys.path.insert(0, project_root)

# Importa a função authenticate_user APÓS carregar as variáveis de ambiente
try:
    from app.auth.auth_service import authenticate_user
    logger.info("Função 'authenticate_user' importada com sucesso.")
except ImportError as e:
    logger.error(f"Erro ao importar 'authenticate_user': {e}")
    logger.error("Verifique se o arquivo 'app/auth/auth_service.py' existe e
está correto.")
    sys.exit(1)
except Exception as e:
```

```
    logger.error(f"Erro inesperado ao importar 'authenticate_user': {e}",
exc_info=True)
    sys.exit(1)

def run_authenticate_user_test():
    """
    Executa um teste direto da função authenticate_user() com credenciais
fornecidas pelo usuário.
    """
    logger.info("\n--- Iniciando teste da função authenticate_user() ---")

    test_email = input("Por favor, digite o email do usuário para teste: ")
    test_password = input("Por favor, digite a senha do usuário para teste: ")

    if not test_email or not test_password:
        logger.warning("Email ou senha não fornecidos. Encerrando teste.")
        return

    try:
        logger.info(f"Chamando authenticate_user() para o email: {test_email}")
        user_data = authenticate_user(test_email, test_password)

        logger.info("☑ authenticate_user() executada com sucesso!")
        logger.info("Dados do usuário autenticado:")
        for key, value in user_data.items():
            logger.info(f"  {key}: {value}")

    except ValueError as e:
        logger.error(f"❌ Falha na autenticação (ValueError): {e}")
        logger.error("Isso geralmente indica credenciais inválidas ou problema
de comunicação com o Supabase.")
    except Exception as e:
        logger.error(f"❌ Ocorreu um erro inesperado ao chamar
authenticate_user(): {e}", exc_info=True)
        logger.error("Verifique o traceback acima para mais detalhes.")

    logger.info("--- Teste da função authenticate_user() concluído ---")

if __name__ == "__main__":
    run_authenticate_user_test()
==================================================

Arquivo: backend\test_connection.py
Conteúdo:
import os
from app.database import supabase
from postgrest.exceptions import APIError


def validar_conexao():
    print("🔍 Iniciando validação de conexão...")

    # 1. Verificar se as variáveis foram carregadas
    url = os.environ.get("SUPABASE_URL")
```

```
    if url:
        print(f"☑ SUPABASE_URL encontrada: {url[:20]}...")
    else:
        print("❌ Erro: SUPABASE_URL não encontrada no ambiente.")
        return

    # 2. Testar chamada simples ao banco de dados
    try:
        # Tenta listar as tabelas ou fazer um select simples na tabela 'users'
        # (Ajuste o nome da tabela se necessário)
        response = supabase.table("users").select("count",
count="exact").limit(1).execute()
        print("☑ Conexão com o Banco de Dados: OK")
        print(f"📊 Total de usuários registrados: {response.count}")

    except APIError as e:
        print(f"❌ Erro de API do Supabase: {e.message}")
    except Exception as e:
        print(f"❌ Erro inesperado: {str(e)}")


if __name__ == "__main__":
    validar_conexao()
==================================================

Arquivo: backend\test_integration.py
Conteúdo:
import asyncio
import httpx
import sys
import os
from pathlib import Path
from dotenv import load_dotenv

# --- CONFIGURAÇÃO ---
BASE_URL = "http://localhost:8080"

# Carrega ambiente para pegar credenciais de teste se existirem
basedir = Path(__file__).resolve().parent
load_dotenv(dotenv_path=basedir / ".env")

# ⚠ AJUSTE ESTAS CREDENCIAIS COM UM DOS 3 USUÁRIOS DO SEU SUPABASE
TEST_EMAIL = os.environ.get("TEST_USER_EMAIL", "ffasti01@gmail.com")
TEST_PASSWORD = os.environ.get("TEST_USER_PASSWORD", "123456")


async def run_integration_test():
    async with httpx.AsyncClient(timeout=30.0) as client:
        print("\n" + "=" * 60)
        print("🚀 INICIANDO TESTE DE INTEGRAÇÃO - DR. TILÁPIA 2.0")
        print("=" * 60)

        # 1. Teste de Health Check
        print("\n[1/3] Verificando integridade da API (/health)...")
```

```
        try:
            res_health = await client.get(f"{BASE_URL}/health")
            if res_health.status_code == 200:
                print(f"☑ API Online: {res_health.json()}")
            else:
                print(f"❌ API Offline ou erro: {res_health.status_code}")
                return
        except Exception as e:
            print(f"❌ Erro de conexão: {e}")
            return

        # 2. Teste de Login
        print(f"\n[2/3] Tentando autenticação para: {TEST_EMAIL}...")
        login_data = {"email": TEST_EMAIL, "password": TEST_PASSWORD}
        res_login = await client.post(f"{BASE_URL}/auth/login", json=login_data)

        if res_login.status_code == 200:
            auth_data = res_login.json()
            token = auth_data.get("access_token")
            user_name = auth_data.get("user_name")
            print(f"☑ Login bem-sucedido! Bem-vindo, {user_name}.")
            print(f"🔑 Token JWT obtido (parcial): {token[:20]}...")
        else:
            print(f"❌ Falha no login ({res_login.status_code}):
{res_login.json().get('detail')}")
            return

        # 3. Teste de Chat RAG (Protegido)
        print("\n[3/3] Enviando pergunta técnica ao Consultor IA
(/consultoria/chat)...")
        headers = {"Authorization": f"Bearer {token}"}
        chat_data = {
            "message": "Qual a temperatura ideal para a criação de tilápias?",
            "history": []
        }

        try:
            res_chat = await client.post(
                f"{BASE_URL}/consultoria/chat",
                json=chat_data,
                headers=headers
            )

            if res_chat.status_code == 200:
                answer = res_chat.json()
                print("\n☑ RESPOSTA DA IA RECEBIDA:")
                print("-" * 40)
                print(answer.get("answer"))
                print("-" * 40)
                print("\n📚 Fontes:")
                for src in answer.get("sources", []):
                    print(f"  - {Path(src).name}")
            else:
                print(f"❌ Erro no Chat ({res_chat.status_code}):
```

```
{res_chat.json().get('detail')}")
        except Exception as e:
            print(f"❌ Erro durante a consulta RAG: {e}")

        print("\n" + "=" * 60)
        print("▩ TESTE DE INTEGRAÇÃO FINALIZADO")
        print("=" * 60)


if __name__ == "__main__":
    if TEST_EMAIL == "seu_email@exemplo.com":
        print("⚠  AVISO: Edite o arquivo test_integration.py com um
e-mail/senha válidos do seu Supabase.")
    else:
        asyncio.run(run_integration_test())
==================================================

Arquivo: backend\test_login_request.py
Conteúdo:
import os
import sys
import logging
import traceback
from dotenv import load_dotenv

# Configuração de Logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -
%(message)s')
logger = logging.getLogger(__name__)

def run_simulated_login_test():
    """
    Simula uma requisição POST para /auth/login e testa a função
authenticate_user()
    com os dados que o backend receberia.
    """
    logger.info("\n--- Iniciando teste de simulação de requisição de login ---")

    # 1. Carregar variáveis de ambiente
    logger.info("Passo 1: Carregando variáveis de ambiente do .env...")
    load_dotenv()
    logger.info("Variáveis de ambiente carregadas.")

    # Adiciona o diretório raiz do projeto ao sys.path
    current_dir = os.path.dirname(os.path.abspath(__file__))
    project_root = os.path.abspath(os.path.join(current_dir, os.pardir))
    if project_root not in sys.path:
        sys.path.insert(0, project_root)
    logger.info(f"sys.path ajustado para incluir: {project_root}")

    # 2. Importar a função authenticate_user
    logger.info("Passo 2: Tentando importar 'authenticate_user' de
'app.auth.auth_service'...")
    try:
```

```
        from app.auth.auth_service import authenticate_user
        logger.info("Função 'authenticate_user' importada com sucesso.")
    except ImportError as e:
        logger.error(f"Erro de importação: {e}")
        logger.error("Verifique se o arquivo 'app/auth/auth_service.py' existe e
está no caminho correto.")
        sys.exit(1)
    except Exception as e:
        logger.error(f"Erro inesperado ao importar 'authenticate_user': {e}",
exc_info=True)
        sys.exit(1)

    # 3. Simular uma requisição POST com email e password
    print("\n--- Dados da Requisição Simulada ---")
    simulated_email = input("Por favor, digite o email para a requisição
simulada: ")
    simulated_password = input("Por favor, digite a senha para a requisição
simulada: ")

    if not simulated_email or not simulated_password:
        logger.warning("Email ou senha não fornecidos. Encerrando teste.")
        return

    logger.info(f"\nPasso 3: Simulando requisição POST para /auth/login com:")
    logger.info(f"  Email: {simulated_email}")
    logger.info(f"  Senha: (não exibida por segurança)")

    # 4. Chamar authenticate_user() com os MESMOS dados que o backend receberia
    logger.info(f"\nPasso 4: Chamando authenticate_user() com os dados
simulados...")
    try:
        user_data = authenticate_user(simulated_email, simulated_password)

        # 5. Exibir o resultado (sucesso)
        logger.info("☑ authenticate_user() executada com sucesso com dados
simulados!")
        logger.info("Dados do usuário autenticado:")
        for key, value in user_data.items():
            logger.info(f"  {key}: {value}")

    except Exception as e:
        # 5. Exibir o resultado (erro detalhado)
        logger.error(f"❌ Ocorreu uma exceção durante a autenticação com dados
simulados!")
        logger.error(f"Tipo da exceção: {type(e).__name__}")
        logger.error(f"Mensagem da exceção: {e}")
        logger.error("Traceback completo:")
        traceback.print_exc(file=sys.stderr)
        logger.error("Isso indica um problema na função authenticate_user() ou
na comunicação com o Supabase.")

    logger.info("\n--- Teste de simulação de requisição de login concluído ---")

if __name__ == "__main__":
```

```
    run_simulated_login_test()
==================================================

Arquivo: backend\test_rag.py
Conteúdo:
import asyncio
import os
import logging
import sys
from pathlib import Path
from dotenv import load_dotenv

# --- PASSO 1: CARREGAR AMBIENTE IMEDIATAMENTE ---
# Isso garante que a OPENAI_API_KEY esteja disponível antes de importar o
serviço
basedir = Path(__file__).resolve().parent
env_path = basedir / '.env'
load_dotenv(dotenv_path=env_path)

# --- PASSO 2: AJUSTE DE PYTHONPATH E IMPORTAÇÃO ---
# Garante que o Python encontre a pasta 'app' mesmo rodando de diretórios
diferentes
if str(basedir) not in sys.path:
    sys.path.append(str(basedir))

try:
    from app.services.rag_service import rag_service
except ImportError as e:
    print(f"❌ Erro de importação: {e}")
    print(f"Current sys.path: {sys.path}")
    sys.exit(1)

# Configuração de logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("TestRAG")


async def testar_rag():
    print("\n" + "=" * 50)
    print("🔍 INICIANDO VALIDAÇÃO DO SERVIÇO RAG")
    print("=" * 50)

    # Verificação rápida da chave
    api_key = os.environ.get("OPENAI_API_KEY")
    if not api_key:
        print("❌ ERRO: OPENAI_API_KEY não encontrada no arquivo .env")
        return
    print(f"☑ Chave OpenAI carregada: {api_key[:8]}...")

    # Pergunta baseada no contexto do projeto Dr. Tilápia
    pergunta = "Como funciona o sistema de monitoramento do Dr. Tilápia?"
    historico = []

    print(f"\n❓ Pergunta: {pergunta}")
```

```
    print("⏳ Aguardando resposta da IA...")

    try:
        resultado = await rag_service.get_response(pergunta, historico)

        print("\n✅ RESPOSTA GERADA:")
        print("-" * 30)
        print(resultado["answer"])
        print("-" * 30)

        print("\n📚 FONTES UTILIZADAS:")
        if resultado["sources"]:
            for source in resultado["sources"]:
                # Exibe apenas o nome do arquivo para limpeza visual
                print(f"  • {Path(source).name}")
        else:
            print("  ⚠️ Nenhuma fonte encontrada. Verifique se há PDFs em
'backend/docs/'.")

    except Exception as e:
        print(f"❌ ERRO DURANTE A EXECUÇÃO: {str(e)}")


if __name__ == "__main__":
    # Executa o loop de eventos assíncronos
    asyncio.run(testar_rag())
==================================================
Diretório: backend\app
--------------------------------------------------

Arquivo: backend\app\__init__.py
Conteúdo:
"""
Pacote principal da aplicação DrTilápia 1.3.
Centraliza importações de módulos críticos.
"""

__version__ = "2.0"
==================================================

Arquivo: backend\app\database.py
Conteúdo:
import os
from pathlib import Path
from supabase import create_client, Client
from dotenv import load_dotenv

# Configuração de caminhos para evitar problemas com OneDrive/Windows
basedir = Path(__file__).resolve().parent.parent
env_path = basedir / '.env'
load_dotenv(dotenv_path=env_path)

url: str = os.environ.get("SUPABASE_URL")
key: str = os.environ.get("SUPABASE_KEY")
```

```
if not url or not key:
    # Fallback para diretório atual
    load_dotenv()
    url = os.environ.get("SUPABASE_URL")
    key = os.environ.get("SUPABASE_KEY")

# Objeto global do Supabase utilizado por todo o sistema
supabase: Client = create_client(url, key)
==================================================
Diretório: backend\app\auth
--------------------------------------------------

Arquivo: backend\app\auth\__init__.py
Conteúdo:
"""
Módulo de autenticação do DrTilápia 2.0.
"""
==================================================

Arquivo: backend\app\auth\auth_service.py
Conteúdo:
import jwt
import os
from datetime import datetime, timedelta
from passlib.context import CryptContext
from email_validator import validate_email, EmailNotValidError
import logging
from typing import Optional

# Configuração de Logging
logger = logging.getLogger(__name__)

# --- Configurações de Segurança ---
SECRET_KEY = os.environ.get("SECRET_KEY",
"uma_chave_secreta_muito_forte_e_aleatoria_para_o_DrTilapia_1.3")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_HOURS = 24

# Contexto para hashing de senhas (bcrypt)
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")


# --- Funções de Validação ---

def validar_email(email: str) -> bool:
    """
    Valida o formato de um endereço de e-mail usando a biblioteca
email_validator.

    Args:
        email (str): O endereço de e-mail a ser validado.

    Returns:
```

```
        bool: True se o e-mail for válido, False caso contrário.
    """
    try:
        validate_email(email)
        return True
    except EmailNotValidError as e:
        logger.warning(f"Validação de e-mail falhou para '{email}': {e}")
        return False
    except Exception as e:
        logger.error(f"Erro inesperado na validação de e-mail para '{email}':
{e}")
        return False


def validar_senha(senha: str) -> bool:
    """
    Valida a força de uma senha com base em critérios mínimos.

    Critérios:
    - Mínimo de 8 caracteres.
    - Pelo menos uma letra maiúscula.
    - Pelo menos um dígito.
    - Pelo menos um caractere especial.

    Args:
        senha (str): A senha a ser validada.

    Returns:
        bool: True se a senha atender aos critérios de força, False caso
contrário.
    """
    if len(senha) < 8:
        logger.warning("Validação de senha falhou: Senha muito curta (mínimo 8
caracteres).")
        return False
    if not any(c.isupper() for c in senha):
        logger.warning("Validação de senha falhou: Senha deve conter pelo menos
uma letra maiúscula.")
        return False
    if not any(c.isdigit() for c in senha):
        logger.warning("Validação de senha falhou: Senha deve conter pelo menos
um dígito.")
        return False

    special_characters = "!@#$%^&*()_+-=[]{}|;:,.<>?"
    if not any(c in special_characters for c in senha):
        logger.warning("Validação de senha falhou: Senha deve conter pelo menos
um caractere especial.")
        return False
    return True


# --- Funções de Hashing de Senha ---
```

```
def get_password_hash(password: str) -> str:
    """
    Gera o hash de uma senha usando bcrypt.

    Args:
        password (str): A senha em texto puro.

    Returns:
        str: O hash da senha.
    """
    return pwd_context.hash(password)


def verify_password(plain_password: str, hashed_password: str) -> bool:
    """
    Verifica se uma senha em texto puro corresponde a um hash de senha.

    Args:
        plain_password (str): A senha em texto puro fornecida pelo usuário.
        hashed_password (str): O hash da senha armazenado.

    Returns:
        bool: True se as senhas corresponderem, False caso contrário.
    """
    try:
        return pwd_context.verify(plain_password, hashed_password)
    except Exception as e:
        logger.error(f"Erro ao verificar senha: {e}")
        return False


# --- Funções JWT (JSON Web Token) ---

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None)
-> str:
    """
    Cria um token de acesso JWT.

    Args:
        data (dict): Dados a serem codificados no token (ex: {"sub":
"user@example.com"}).
        expires_delta (Optional[timedelta]): Tempo de expiração do token. Se
None, usa o padrão.

    Returns:
        str: O token JWT codificado.
    """
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() + timedelta(hours=ACCESS_TOKEN_EXPIRE_HOURS)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

```
    logger.info(f"Token JWT criado para dados: {data.get('sub', 'N/A')}")
    return encoded_jwt


def verify_access_token(token: str) -> Optional[dict]:
    """
    Verifica e decodifica um token de acesso JWT.

    Args:
        token (str): O token JWT a ser verificado.

    Returns:
        Optional[dict]: O payload do token se for válido, None caso contrário.
    """
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        logger.info(f"Token JWT verificado com sucesso. Payload:
{payload.get('sub', 'N/A')}")
        return payload
    except jwt.ExpiredSignatureError:
        logger.warning("Verificação de token falhou: Token expirado.")
        return None
    except jwt.InvalidTokenError:
        logger.warning("Verificação de token falhou: Token inválido.")
        return None
    except Exception as e:
        logger.error(f"Erro inesperado na verificação de token: {e}")
        return None
==================================================
Diretório: backend\app\auth\__pycache__
--------------------------------------------------
Diretório: backend\app\services
--------------------------------------------------

Arquivo: backend\app\services\__init__.py
Conteúdo:

==================================================

Arquivo: backend\app\services\rag_service.py
Conteúdo:
import os
import logging
from pathlib import Path
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_community.document_loaders import PyPDFLoader, DirectoryLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.chains import ConversationalRetrievalChain

# --- CONFIGURAÇÃO DE AMBIENTE (ROBUSTA) ---
# Resolve o caminho do .env subindo 3 níveis (services -> app -> backend ->
.env)
```

```
basedir = Path(__file__).resolve().parent.parent.parent
env_path = basedir / '.env'
load_dotenv(dotenv_path=env_path)

# Configuração de Logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("RAGService")


class RAGService:
    def __init__(self):
        # Agora o ambiente já está carregado, evitando o erro de api_key
        self.embeddings = OpenAIEmbeddings()
        self.vectorstore_path = str(basedir / "vectorstore_faiss")
        self.docs_path = str(basedir / "docs")
        self._vectorstore = None

    def _load_vectorstore(self):
        """Carrega o FAISS local ou cria um novo a partir dos PDFs."""
        index_file = Path(self.vectorstore_path) / "index.faiss"

        if index_file.exists():
            logger.info(f"Carregando VectorStore existente em:
{self.vectorstore_path}")
            return FAISS.load_local(
                self.vectorstore_path,
                self.embeddings,
                allow_dangerous_deserialization=True
            )

        logger.info("Criando novo VectorStore a partir dos documentos em
/docs...")
        if not os.path.exists(self.docs_path):
            os.makedirs(self.docs_path)

        loader = DirectoryLoader(self.docs_path, glob="*.pdf",
loader_cls=PyPDFLoader)
        documents = loader.load()

        if not documents:
            logger.warning(f"Nenhum documento encontrado em {self.docs_path}.")
            return None

        text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
chunk_overlap=200)
        chunks = text_splitter.split_documents(documents)

        vectorstore = FAISS.from_documents(chunks, self.embeddings)
        vectorstore.save_local(self.vectorstore_path)
        return vectorstore

    async def get_response(self, question: str, chat_history: list):
        """Gera resposta baseada nos documentos e histórico."""
        if self._vectorstore is None:
```

```
            self._vectorstore = self._load_vectorstore()

        if self._vectorstore is None:
            return {
                "answer": "A base de conhecimento está vazia. Adicione PDFs na
pasta /docs.",
                "sources": []
            }

        chain = ConversationalRetrievalChain.from_llm(
            llm=ChatOpenAI(model="gpt-4-turbo", temperature=0),
            retriever=self._vectorstore.as_retriever(search_kwargs={"k": 3}),
            return_source_documents=True
        )

        # LangChain invoke espera o dicionário com as chaves corretas
        result = chain.invoke({"question": question, "chat_history":
chat_history})

        # Extração única das fontes
        sources = list(set([doc.metadata.get("source") for doc in
result["source_documents"]]))

        return {
            "answer": result["answer"],
            "sources": sources
        }


# Instância global para o projeto
rag_service = RAGService()
==================================================
Diretório: backend\app\services\__pycache__
--------------------------------------------------
Diretório: backend\app\__pycache__
--------------------------------------------------
Diretório: backend\docs
--------------------------------------------------
Diretório: backend\vectorstore_faiss
--------------------------------------------------
Diretório: backend\__pycache__
--------------------------------------------------
Diretório: frontend
--------------------------------------------------

Arquivo: frontend\rxconfig.py
Conteúdo:
import reflex as rx

config = rx.Config(
    app_name="ui",
)
==================================================
Diretório: frontend\assets
```

```
--------------------------------------------------
Diretório: frontend\ui
--------------------------------------------------

Arquivo: frontend\ui\__init__.py
Conteúdo:

==================================================

Arquivo: frontend\ui\state.py
Conteúdo:
import reflex as rx
import httpx
import logging
from typing import List, Tuple

BACKEND_URL = "http://localhost:8080"
logger = logging.getLogger(__name__)

class State(rx.State):
    user_email: str = ""
    password: str = ""
    user_name: str = "Usuário"
    user_role: str = rx.LocalStorage("user")
    access_token: str = rx.LocalStorage("")
    is_authenticated: bool = False
    current_message: str = ""
    chat_history: List[Tuple[str, str]] = []
    is_loading: bool = False
    error_message: str = ""

    # Setters explícitos para evitar DeprecationWarnings
    def set_user_email(self, val: str): self.user_email = val
    def set_password(self, val: str): self.password = val
    def set_current_message(self, val: str): self.current_message = val

    def check_login(self):
        if not self.access_token: return rx.redirect("/login")
        self.is_authenticated = True

    def logout(self):
        self.access_token = ""
        self.is_authenticated = False
        self.user_role = "user"
        return rx.redirect("/")

    async def handle_login(self):
        self.is_loading = True
        try:
            async with httpx.AsyncClient() as client:
                res = await client.post(f"{BACKEND_URL}/auth/login",
json={"email": self.user_email, "password": self.password})
                if res.status_code == 200:
                    data = res.json()
```

```
                    self.access_token = data["access_token"]
                    self.user_name = data["user_name"]
                    self.user_role = data["user_role"]
                    return rx.redirect("/hub")
                self.error_message = "Credenciais inválidas."
        except: self.error_message = "Erro de conexão."
        finally: self.is_loading = False

    async def handle_chat(self):
        """Método que estava faltando ou com erro de referência."""
        if not self.current_message: return
        msg = self.current_message
        self.chat_history.append((msg, ""))
        self.current_message = ""
        self.is_loading = True
        try:
            async with httpx.AsyncClient() as client:
                headers = {"Authorization": f"Bearer {self.access_token}"}
                res = await client.post(
                    f"{BACKEND_URL}/consultoria/chat",
                    json={"message": msg, "history": [list(p) for p in
self.chat_history[:-1]]},
                    headers=headers, timeout=60.0
                )
                self.chat_history[-1] = (msg, res.json().get("answer", "Sem
resposta."))
        except:
            self.chat_history[-1] = (msg, "Erro na conexão com a IA.")
        finally: self.is_loading = False

    async def handle_upload(self, files: List[rx.UploadFile]):
        self.is_loading = True
        try:
            async with httpx.AsyncClient() as client:
                headers = {"Authorization": f"Bearer {self.access_token}"}
                for file in files:
                    content = await file.read()
                    await client.post(f"{BACKEND_URL}/admin/upload",
files={"file": (file.filename, content)}, headers=headers)
            return [rx.window_alert("Upload concluído!"),
rx.clear_selected_files("upload_manual")]
        except: return rx.window_alert("Falha no upload.")
        finally: self.is_loading = False

    async def handle_reindex(self):
        self.is_loading = True
        try:
            async with httpx.AsyncClient() as client:
                headers = {"Authorization": f"Bearer {self.access_token}"}
                await client.post(f"{BACKEND_URL}/admin/reindex",
headers=headers)
            return rx.window_alert("Reindexado!")
        except: pass
        finally: self.is_loading = False
```

==========

Arquivo: frontend\ui\styles.py
Conteúdo:
```
import reflex as rx

PRIMARY = "#004e92"
ACCENT = "#ff7f50"
SECONDARY = "#00ced1"
WHITE = "#ffffff"

BACKGROUND_GRADIENT = f"radial-gradient(circle at center, {PRIMARY} 0%, #000814
100%)"

HERO_BG_STYLE = {
    "background_image": "url('/hub01.jpeg')",
    "background_size": "cover",
    "background_position": "center",
    "background_repeat": "no-repeat",
    "background_attachment": "fixed",
}

GLASS_CONTAINER_STYLE = {
    "backdrop_filter": "blur(12px)",
    "background_color": "rgba(255, 255, 255, 0.05)",
    "border": "1px solid rgba(255, 255, 255, 0.1)",
    "border_radius": "1.5em",
    "box_shadow": "0 8px 32px 0 rgba(0, 0, 0, 0.37)",
}

BUTTON_ACCENT = {
    "bg": ACCENT,
    "color": WHITE,
    "_hover": {"bg": "#ff6347", "transform": "scale(1.02)"},
}
```
==========

Arquivo: frontend\ui\ui.py
Conteúdo:
```
import reflex as rx
from ui.state import State
from ui.pages.index import index
from ui.pages.login import login_page
from ui.pages.hub import hub
from ui.pages.consultoria import consultoria
from ui.pages.admin_rag import admin_rag
from ui.pages.dashboard import dashboard

app = rx.App(
    theme=rx.theme(appearance="dark", accent_color="blue")
)

# Registro de Rotas
app.add_page(index, route="/")
```

```
app.add_page(login_page, route="/login")
app.add_page(hub, route="/hub", on_load=State.check_login)
app.add_page(consultoria, route="/consultoria", on_load=State.check_login)
app.add_page(admin_rag, route="/admin-rag", on_load=State.check_login)
app.add_page(dashboard, route="/dashboard", on_load=State.check_login)
==================================================
Diretório: frontend\ui\pages
--------------------------------------------------

Arquivo: frontend\ui\pages\admin_rag.py
Conteúdo:
import reflex as rx
from ui.state import State
from ui.styles import GLASS_CONTAINER_STYLE, ACCENT, WHITE, BACKGROUND_GRADIENT,
BUTTON_ACCENT

def admin_rag() -> rx.Component:
    return rx.center(
        rx.vstack(
            rx.heading("Gestão de Conhecimento", color=WHITE, size="8"),
            rx.upload(
                rx.vstack(
                    rx.button("Selecionar PDFs", color=ACCENT,
variant="outline"),
                    rx.text("Ou arraste os manuais aqui", font_size="0.8em",
color="rgba(255,255,255,0.5)"),
                    spacing="2", align="center",
                ),
                id="upload_manual",
                multiple=True,
                accept={"application/pdf": [".pdf"]},
                border=f"2px dashed {ACCENT}", padding="4em", width="100%",
                bg="rgba(255, 255, 255, 0.03)",
            ),
            # Feedback visual do Drag and Drop
            rx.vstack(
                rx.foreach(rx.selected_files("upload_manual"), lambda f:
rx.text(f"📄 {f}", color=WHITE, font_size="0.8em")),
                width="100%", align="start"
            ),
            rx.button("Enviar Arquivos",
on_click=State.handle_upload(rx.upload_files(upload_id="upload_manual")),
is_loading=State.is_loading, width="100%", **BUTTON_ACCENT),
            rx.link("Voltar ao Hub", href="/hub", color=WHITE, opacity="0.6"),
            width="550px", padding="3em", spacing="5", **GLASS_CONTAINER_STYLE
        ),
        height="100vh", width="100%", background=BACKGROUND_GRADIENT
    )
==================================================

Arquivo: frontend\ui\pages\consultoria.py
Conteúdo:
import reflex as rx
from ui.state import State
```

```
from ui.styles import GLASS_CONTAINER_STYLE, ACCENT, WHITE, BACKGROUND_GRADIENT

def message_box(chat_pair):
    return rx.vstack(
        rx.box(rx.text(chat_pair[0], color=WHITE), align_self="flex-end",
bg="rgba(255,255,255,0.1)", padding="1em", border_radius="1em"),
        rx.box(rx.markdown(chat_pair[1]), align_self="flex-start",
bg="rgba(0,206,209,0.1)", padding="1em", border_radius="1em", border=f"1px solid
{ACCENT}33"),
        width="100%", spacing="2"
    )

def consultoria() -> rx.Component:
    return rx.center(
        rx.vstack(
            rx.hstack(
                rx.icon(tag="bot", color=ACCENT),
                rx.heading("Consultor IA", color=WHITE),
                rx.spacer(),
                rx.link("Voltar", href="/hub", color=WHITE),
                width="100%", align="center"
            ),
            rx.scroll_area(
                rx.vstack(rx.foreach(State.chat_history, message_box),
width="100%", spacing="4"),
                height="60vh", width="100%"
            ),
            rx.hstack(
                rx.input(placeholder="Sua dúvida...",
value=State.current_message, on_change=State.set_current_message, width="100%",
color=WHITE),
                rx.button(rx.icon(tag="send"), on_click=State.handle_chat,
is_loading=State.is_loading, bg=ACCENT),
                width="100%"
            ),
            width="800px", padding="2em", **GLASS_CONTAINER_STYLE
        ),
        height="100vh", width="100%", background=BACKGROUND_GRADIENT
    )
==================================================

Arquivo: frontend\ui\pages\dashboard.py
Conteúdo:
import reflex as rx
from ui.styles import GLASS_CONTAINER_STYLE, ACCENT, WHITE, BACKGROUND_GRADIENT,
SECONDARY

def metric(label, val, unit):
    return rx.vstack(
        rx.text(label, font_size="0.8em", color="rgba(255,255,255,0.6)"),
        rx.hstack(rx.text(val, font_size="2em", font_weight="bold",
color=WHITE), rx.text(unit, color=SECONDARY)),
        bg="rgba(0,0,0,0.2)", padding="1.5em", border_radius="1em",
width="180px"
```

```
    )

def dashboard() -> rx.Component:
    return rx.center(
        rx.vstack(
            rx.heading("Métricas em Tempo Real", color=WHITE),
            rx.flex(
                metric("Temperatura", "26.5", "°C"),
                metric("Oxigênio", "7.2", "mg/L"),
                metric("pH", "6.8", "pH"),
                gap="1.5em", flex_wrap="wrap", justify="center"
            ),
            rx.link("Voltar ao Hub", href="/hub", color=WHITE,
margin_top="2em"),
            width="800px", padding="3em", **GLASS_CONTAINER_STYLE
        ),
        height="100vh", width="100%", background=BACKGROUND_GRADIENT
    )
==================================================

Arquivo: frontend\ui\pages\hub.py
Conteúdo:
import reflex as rx
from ui.state import State
from ui.styles import WHITE, ACCENT, GLASS_CONTAINER_STYLE


def floating_card(title: str, desc: str, href: str, icon: str, **kwargs):
    return rx.link(
        rx.vstack(
            rx.icon(tag=icon, size=30, color=ACCENT),
            rx.text(title, font_weight="bold", color=WHITE),
            rx.text(desc, font_size="0.8em", color="rgba(255,255,255,0.6)"),
            spacing="2", align="center",
        ),
        href=href, padding="1.5em", **GLASS_CONTAINER_STYLE,
position="absolute", **kwargs
    )


def hub() -> rx.Component:
    return rx.box(
        rx.image(src="/hub01.jpeg", width="100%", height="100vh",
object_fit="cover", position="absolute", z_index="-1",
                 filter="brightness(0.4)"),
        rx.center(rx.heading(f"Olá, {State.user_name}", size="8", color=WHITE,
position="absolute", top="10%")),

        floating_card("Consultoria", "Chat IA", "/consultoria",
"message-square", top="40%", left="15%"),
        floating_card("Dashboard", "Métricas", "/dashboard", "bar-chart-3",
top="40%", right="15%"),

        rx.cond(
```

```
            State.user_role == "admin",
            floating_card("Admin RAG", "Gestão", "/admin-rag", "database",
bottom="15%", left="43%"),
        ),

        rx.button("Sair", on_click=State.logout, position="absolute", top="2em",
right="2em", variant="ghost",
                  color=WHITE),
        height="100vh", width="100%", position="relative"
    )
```

==========

Arquivo: frontend\ui\pages\index.py
Conteúdo:

```
import reflex as rx
from ui.styles import (
    WHITE,
    BUTTON_ACCENT,
    HERO_BG_STYLE
)


def index() -> rx.Component:
    """Landing Page minimalista com fundo hub01.jpeg."""
    return rx.center(
        rx.vstack(
            # Título e Subtítulo (ImagemPrincipal removida)
            rx.heading(
                "Dr. Tilápia 2.0",
                size="9",
                color=WHITE,
                text_shadow="0 4px 15px rgba(0,0,0,0.6)"
            ),

            rx.text(
                "A inteligência artificial que transforma a sua piscicultura.",
                color="rgba(255,255,255,0.8)",
                font_size="1.4em",
                text_align="center",
                max_width="700px",
                font_weight="medium"
            ),

            rx.spacer(),

            # Botão de Acesso
            rx.link(
                rx.button(
                    "Entrar no Sistema",
                    size="4",
                    padding="2em 4em",
                    **BUTTON_ACCENT
                ),
                href="/login",
```

```
            ),

            spacing="6",
            align="center",
            padding="4em",
            # Overlay escuro para destacar o texto sobre a hub01.jpeg
            bg="rgba(0, 8, 20, 0.55)",
            width="100%",
            height="100vh",
        ),
        # Aplica o fundo hub01.jpeg definido no styles.py
        **HERO_BG_STYLE,
        height="100vh",
        width="100%",
    )
```

==========

Arquivo: frontend\ui\pages\login.py
Conteúdo:

```
import reflex as rx
from ui.state import State
from ui.styles import WHITE, GLASS_CONTAINER_STYLE, BUTTON_ACCENT, PRIMARY

def login_page():
    return rx.center(
        rx.vstack(
            rx.heading("Dr. Tilápia 2.0", size="8", color=WHITE),
            rx.input(placeholder="E-mail", on_change=State.set_user_email,
width="100%", bg="rgba(255,255,255,0.05)"),
            rx.input(placeholder="Senha", type="password",
on_change=State.set_password, width="100%", bg="rgba(255,255,255,0.05)"),
            rx.button("Entrar", on_click=State.handle_login,
is_loading=State.is_loading, width="100%", **BUTTON_ACCENT),
            rx.text(State.error_message, color="red"),
            spacing="4", padding="3em", width="400px", **GLASS_CONTAINER_STYLE
        ),
        height="100vh", width="100%", background=f"radial-gradient(circle,
{PRIMARY}, #000)"
    )
```